

Technical Design Document

Autonomous Voice-Agent Orchestration for Background Verification (BGV)

Classification	CONFIDENTIAL — Internal Use Only
Version	MVP 1.0
Runtime	Python 3.11+ · FastMCP · SQLite
Voice Platform	Dograh AI · Twilio Voice API
Network Ingress	Cloudflare Tunnels

MVP Automation Scope

- Dynamic candidate and case state retrieval
- Automated insufficiency classification and management
- Real-time SLA tracking and contact mutation
- Full-duplex outbound voice streaming (Dograh & Twilio)
- Cryptographic and temporal logging of all conversational interactions

1. Executive Summary

1.1 Project Vision

This initiative introduces an **autonomous, AI-driven voice orchestration platform** designed to programmatically resolve Background Verification (BGV) bottlenecks. By deploying a conversational AI runtime with real-time database read/write access, the system autonomously identifies candidates with unresolved insufficiencies, initiates contextual voice interactions, and mutates case states deterministically — drastically reducing manual analyst overhead and accelerating onboarding lifecycles.

1.2 Problem Statement

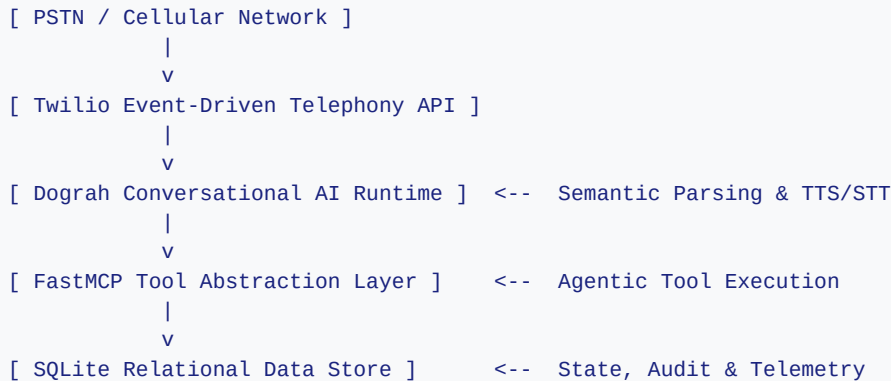
Current BGV workflows suffer from high operational latency due to manual follow-ups on incomplete or insufficient records. This asynchronous human-in-the-loop dependency creates severe onboarding bottlenecks. The proposed architecture replaces manual outreach with an autonomous agentic workflow, enabling real-time data collection, semantic intent parsing, and instant case updates over standard telephony infrastructure.

1.3 Strategic Objectives

- **Proactive Anomaly Detection:** Continuously monitor the data layer to detect unresolved verification insufficiencies.
- **Automated Telephony Outreach:** Trigger programmatic outbound calling pipelines without human intervention.
- **Stateful Conversational AI:** Equip the voice agent with dynamic contextual bridging to conduct highly specific, candidate-aware interviews.
- **Immutable Auditability:** Enforce strict compliance through comprehensive conversation telemetry and state-change audit trails.

2. Architectural Blueprint

2.1 High-Level Topology



2.2 Core Microservices

- **Asynchronous Outbound Engine:** A background polling daemon that monitors BGV case states and triggers outbound webhook payloads to the telephony layer.
- **Telephony Gateway (Twilio):** Handles SIP/PSTN bridging, orchestrating the physical phone call and routing full-duplex audio streams to the AI backend.
- **AI Voice Runtime (Dograh):** The core conversational engine responsible for turn-taking, semantic understanding, and triggering structured function calls based on natural language inputs.
- **Stateful Tool Server (FastMCP):** An abstraction layer that exposes strict, type-safe endpoints allowing the LLM to securely query and mutate backend databases.
- **Persistence Layer (SQLite):** The centralized relational datastore containing candidate profiles, verification logic, session telemetry, and immutable audit logs.

2.2.6 Low-Latency Memory Fabric & Message Broker (Redis Tier)

To meet the **sub-500ms target response threshold** required for natural human conversation, the architecture incorporates an in-memory Redis data tier. This component serves two architectural imperatives:

Real-Time Session State Persistence	Avoids disk-bound I/O operations by caching active dialogue trees, candidate tokens, and conversational context windows directly in memory for instantaneous retrieval during a live call routing cycle.
High-Concurrency Pub/Sub Audio Streaming	Acts as the high-throughput message broker that handles frame-by-frame audio payload distribution, allowing the system to scale horizontally across multiple concurrent telephony channels without blocking the primary event loop.

2.3 Technology Stack

Runtime	Python 3.11+
Tooling Protocol	FastMCP (Model Context Protocol)
Persistence	SQLite (with strict foreign-key constraints)
Voice Orchestration	Dograh AI Platform
Telephony	Twilio Voice API
Network Ingress	Cloudflare Tunnels (Secure Webhook Exposure)

3. Stateful Orchestration & Data Design

3.1 Event-Driven Data Flow

1. Polling Phase	The background engine scans for candidate records flagged with INSUFFICIENCY_RAISED.
2. Dispatch	Webhooks are dispatched to Twilio to initiate the SIP call to the candidate's mobile device.
3. Context Injection	Upon connection, Dograh queries the FastMCP server to inject the specific candidate's insufficiency details into the LLM's active prompt window.
4. Dynamic Interaction	The voice agent converses with the candidate, dynamically requesting clarifications or updated information.
5. State Mutation	The agent triggers FastMCP update tools to instantly write the candidate's verbal confirmations back to the database.
6. Telemetry Archival	The session is terminated, and a complete chronological transcript and state-change audit log are committed to storage.

3.2 Relational Data Topology

The system maintains strict referential integrity across the following entity relationships:

- 1:N Candidate → Verification Components
- 1:N Verification Component → Insufficiencies
- 1:N Candidate → Conversation Sessions
- 1:N Candidate → Immutable Audit Records

4. Agentic Interface & Tooling (MCP)

The voice agent interacts with the backend strictly through **deterministic, schema-validated tool calls**. All endpoints are exposed via the FastMCP abstraction layer.

4.1 Context Retrieval — Read Operations

Tool / Endpoint	Description
<code>get_candidate</code>	Fetches baseline demographic and application data for the candidate.
<code>get_onboarding_status</code>	Retrieves aggregate verification progress across all components.
<code>get_insufficiencies</code>	Isolates the exact documents or data points blocking approval.

4.2 State Mutation — Update Operations

Tool / Endpoint	Description
<code>resolve_insufficiency</code>	Flips boolean flags upon successful verbal data collection.
<code>mark_resubmission_promised</code>	Updates candidate SLAs based on verbal commitments made during the call.
<code>update_candidate_contact</code>	Mutates primary outreach channels (phone / email).
<code>escalate_to_analyst</code>	Flags the case for human-in-the-loop intervention if the AI detects complex edge cases.

4.3 Telemetry & Session Management

Tool / Endpoint	Description
<code>start_session</code> / <code>end_session</code>	Initializes and terminates the conversational envelope.
<code>log_turn</code>	Captures user/agent transcripts and network latency metrics frame-by-frame.

5. Enterprise Security, QA & Future Roadmap

5.1 Security Posture

- **State-Change Auditing:** Every database mutation triggers an automatic, time-stamped entry in the `audit_trails` table.
- **Secrets Management:** Cryptographic API keys (Twilio, Dograh) are injected exclusively via restricted environment variables.

5.2 Quality Assurance — Current Scope

The pipeline has undergone rigorous end-to-end validation across:

- Type-safe MCP tool execution under simulated load
- Graceful handling of dropped calls and incomplete insufficiency resolutions
- Database CRUD atomicity and foreign-key enforcement

5.3 Production Scaling Roadmap

Containerization	Full Dockerization of the FastMCP server and polling workers.
Database Migration	Upgrading the persistence layer from SQLite to a highly available PostgreSQL cluster.
Observability	Integration with Datadog / Prometheus for centralized log aggregation and latency monitoring.
Multilingual Support	Upgrading the Dograh TTS/STT pipelines to support localized Indian dialects for broader candidate reach.